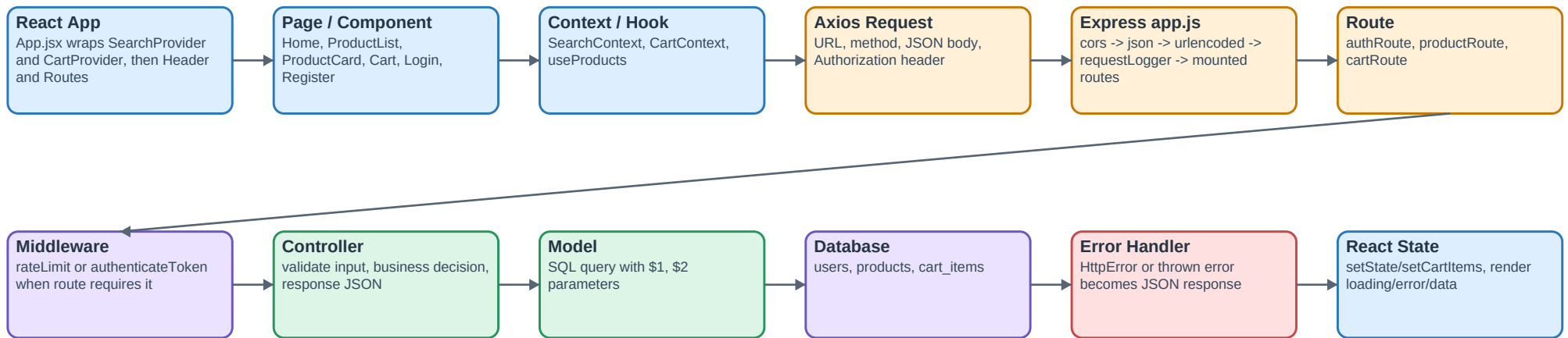


Deep End-to-End Architecture

Request lifecycle from React click to PostgreSQL and back



Core rule

- Frontend only knows the JWT string in localStorage. It does not know the database user id directly.
- Backend verifies the JWT in authenticateToken. After verification, req.user contains the decoded payload.
- Cart controller reads Number(req.user.sub). That value becomes user_id in cart_items.
- Every controller either sends a success JSON response or throws HttpError; errorHandler formats failures.

API Routes and JSON Response Map

All important endpoints with request body, success JSON, and common failures

POST /auth/register	{ username, email, password }	201 { success:true, message, user, token }	409 user exists, 422 validation
POST /auth/login	{ email, password }	200 { message:'Login successful', token }	401 user/password, 429 rate limit, 422 validation
GET /products	none	200 { message, count, products:[...] }	500 database/server
GET /products/filter	query: name/category/brand/price/stock/rating/inStock	200 { message, count, products:[...] }	404 no matches, 422 invalid query
GET /products/:id	path id	200 { message, products: product }	404 not found, 422 invalid id
POST /products	Bearer token + product JSON	201 { message, product }	401 auth, 422 body
PATCH /products/:id	Bearer token + partial product	200 { message, product }	401 auth, 404, 422
GET /cart	Bearer token	200 { items:[products with quantity] }	401 missing/expired/invalid token
POST /cart	Bearer token + { productId }	200 { items:[...] }	401 auth, 404 product, 422 body
DELETE /cart/:productId	Bearer token + path productId	200 { items:[...] }	401 auth, 422 id

Product object shape

```
{
  id, name, price, stock, instock,
  brand, category, rating, quantity?
}
```

quantity appears in cart responses because cartModel joins products with cart_items.

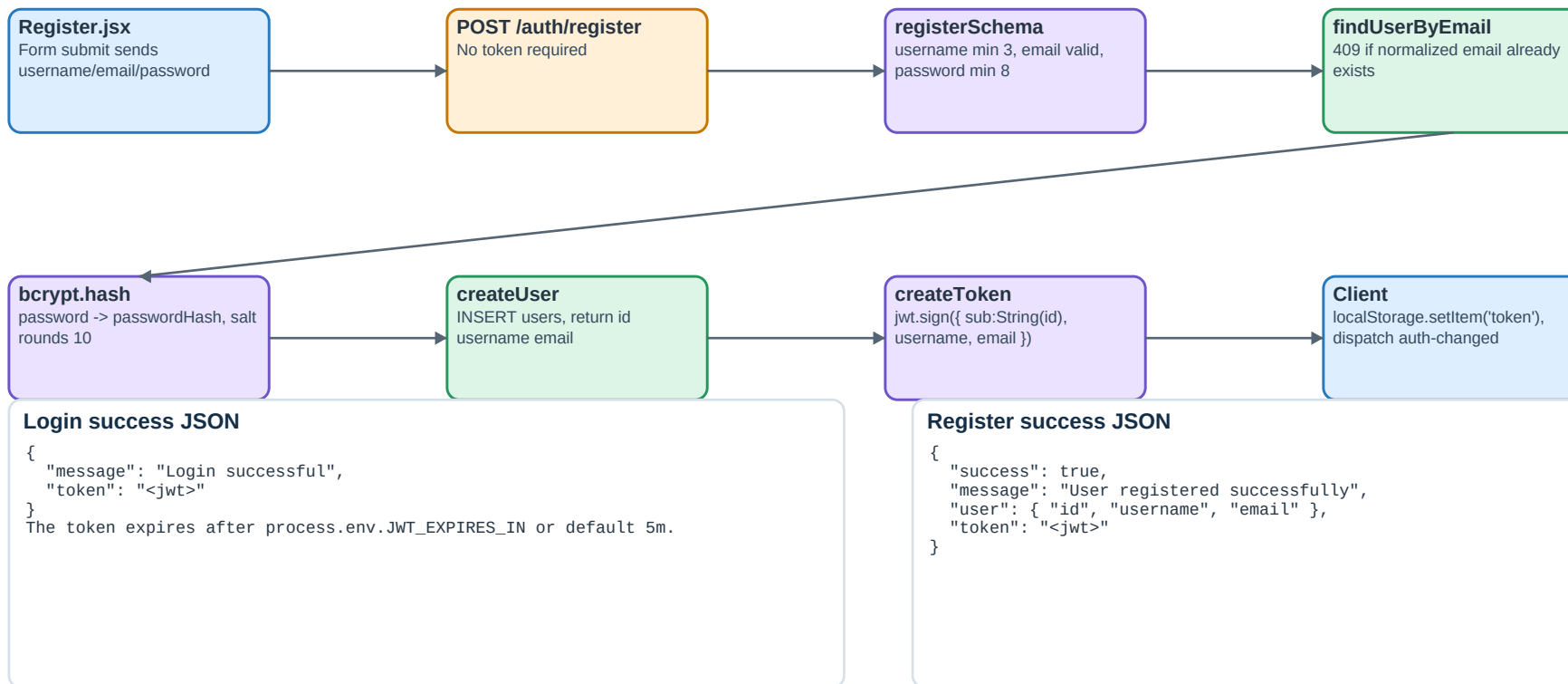
Central error JSON shape

```
{
  "success": false,
  "message": "human readable error"
}
```

Thrown `HttpError/status` errors are normalized by `backend/middlewares/errorHandler.js`.

Authentication Workflow - Deep View

Register and login both finish by creating the same kind of JWT

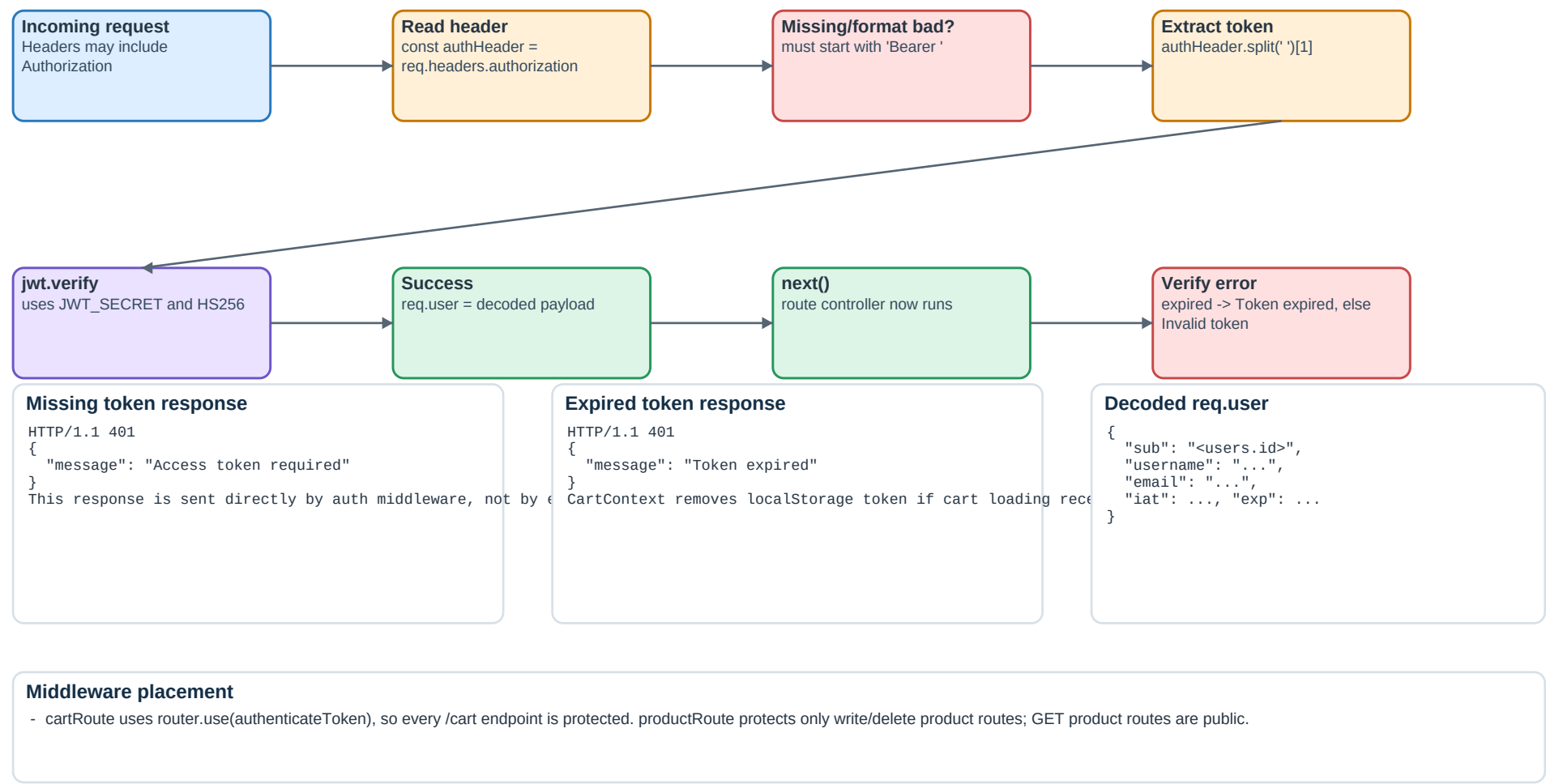


AuthRoute is frontend-only protection

- AuthRoute only checks localStorage.token before showing /cart or /dashboard. Real API security is authenticateToken on the backend. A fake token can pass AuthRoute, but backend rejects it with 401.

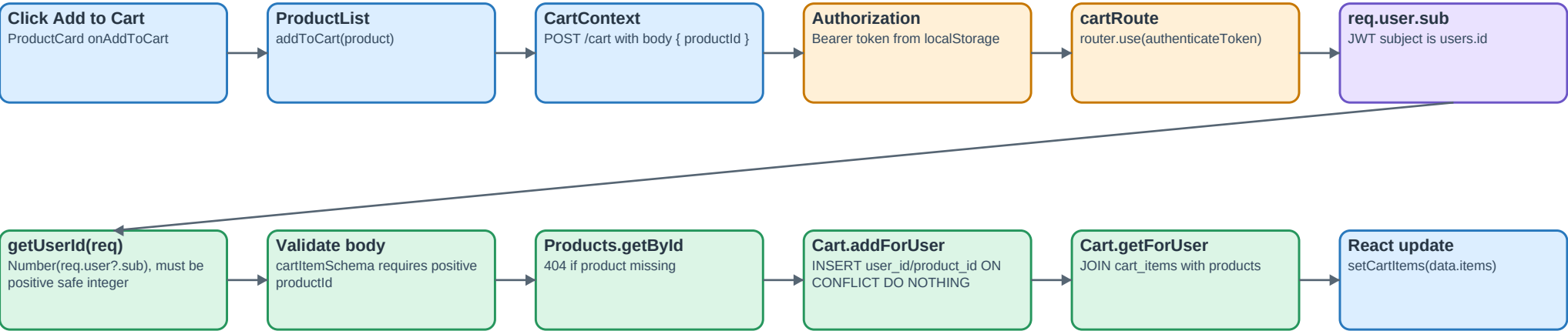
authenticateToken Middleware

Server-side JWT gate used by cart routes and protected product write routes



Cart Workflow - Add, Load, Remove

The cart user is always derived from req.user.sub after JWT verification



POST /cart request

POST /cart
Authorization: Bearer <jwt>
Content-Type: application/json

```
{ "productId": 3 }
```

POST /cart success

```
HTTP/1.1 200  
{  
  "items": [  
    { product fields..., "quantity": 1 }  
  ]  
}
```

DELETE /cart/:productId

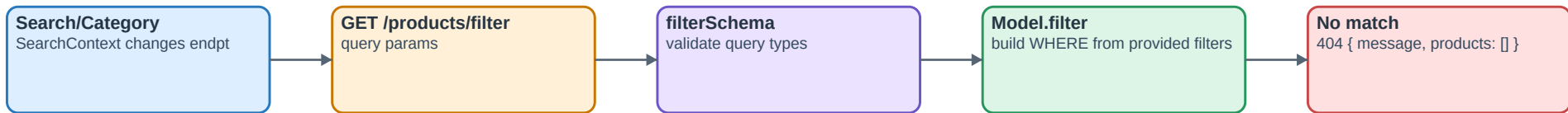
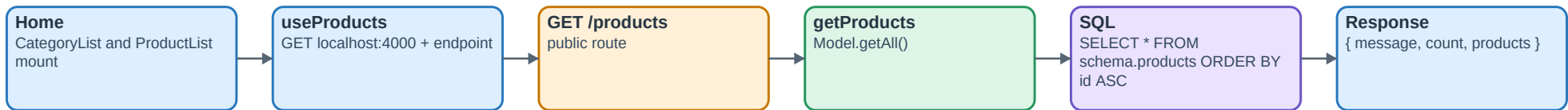
- Same auth path. Controller validates req.params.productId, calls Cart.removeForUser(userId, productId), then returns { items } from Cart.getForUser(userId).

Cart table behavior

- cart_items primary key is (user_id, product_id), so each product appears once per user. addForUser uses ON CONFLICT DO NOTHING; clicking add again does not increase quantity in the current code.

Products, Search, Filter, and Product Write Flow

Public reads, protected writes, Joi validation, SQL model calls



Product filter example

```
GET /products/filter?name=bag

200 {
  "message": "Product data fetched",
  "count": 2,
  "products": [...]
}
```

Protected write request

```
PATCH /products/5
Authorization: Bearer <jwt>

{ "price": 999, "stock": 4 }
```

Requires authenticateToken first.

Write validation

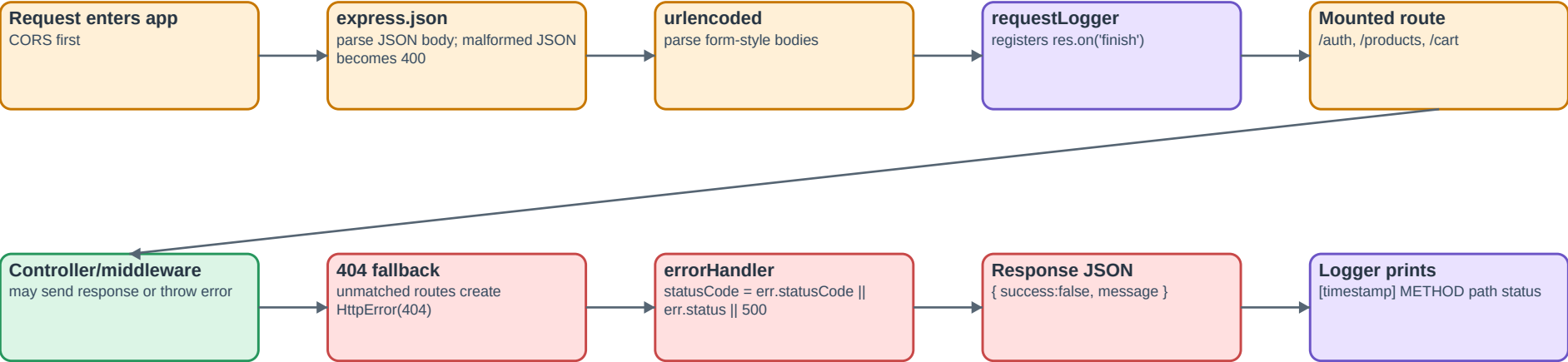
- POST/PUT use productSchema. PATCH uses patchSchema.min(1). id params use idSchema. Validation failure throws HttpError(422).

Important naming detail

- Database column is instock, while request schema uses inStock. Controller/model translate inStock to instock during create/update.

Middleware and Error Handler Deep Flow

How logging, validation errors, unknown routes, and server errors become output



422 validation error
HTTP/1.1 422
{
 "success": false,
 "message": "\"password\" length must be at least 8 characters"
}

404 unknown route
GET /unknown
HTTP/1.1 404
{
 "success": false,
 "message": "Route /unknown not found"
}

Logger output
[2026-10-04T...] GET /products 200
[2026-10-04T...] POST /auth/login 401

It logs after response finish, so it sees final statusCode.

Production behavior

- If status is 500 or above and NODE_ENV is production, errorHandler hides the internal message and returns 'Internal Server Error'. For 4xx errors, it returns the actual message.

Step-by-Step Auth + Cart Mental Model

Use this page to explain the hardest part clearly

1. Login creates identity proof

- User submits email/password from Login.jsx.
- Backend finds the user by lowercased email.
- bcrypt.compare checks password against stored hash.
- jwt.sign creates token with sub equal to database users.id.
- Client stores only the token, not the user id.

2. Cart request sends proof

- CartContext reads token from localStorage.
- GET /cart, POST /cart, DELETE /cart/:productId include Authorization: Bearer <token>.
- If localStorage has no token, CartProvider clears cart state.
- If server returns 401, CartProvider removes the token.

3. Backend finds auth user

- cartRoute runs authenticateToken before every cart controller.
- jwt.verify checks signature, expiry, and algorithm.
- On success, req.user becomes decoded payload.
- getUserId(req) converts req.user.sub to a number.
- Invalid or missing sub throws HttpError(401).

4. Cart model uses that user id

- getUser(userId): SELECT product rows joined to this user's cart_items.
- addForUser(userId, productId): INSERT row for this user/product pair.
- removeForUser(userId, productId): DELETE only this user's row.
- The API never trusts product/user ids from the frontend for user identity.

Final one-line explanation

- Authentication answers: who is making this request? Cart answers: which products belong to that authenticated user? The link between them is the JWT sub claim. After middleware verifies the token, req.user.sub becomes the trusted database user id used in every cart query.